2025.7 - 2025.11 Kaggle

Natural Language Processing with Disaster Tweets

Natural Language Processing with Disaster Tweets 대회 보고서

참가자 : 김동우, 김민석, 이규민, 문병선

목 차

1. Natural Language Processing with Disaster Tweets 개요.....3

2. 데이터 구조.....3

3. 이론적 배경/모델 구현.....6

3.1 LSTM & GRU.....6

3.1.1 LSTM & GRU 모델구현.....7

3.2 Transformer.....10

3.2.1 Transformer 모델구현.....10

3.3 Logistic Regression.....14

3.3.1 Logistic Regression 모델 구현.....14

4. 결론.....19

1. Natural Language Processing with Disaster Tweets 개요

<Natural Language Processing with Disaster Tweets>

트위터는 재난 상황에서 중요한 소통 채널로 활용되고 있다. 사람들은 스마트폰을 통해 자신이 목격한 비상사태를 실시간으로 공유하며, 이에 따라 재난 구호 단체나 통신사 등 여러 기관이 트위터를 체계적으로 모니터링하려는 노력이 늘어나고 있다.

하지만 모든 트윗이 실제 재난을 의미하는 것은 아니다. 예를 들어, "도시가 불타고 있다(Ablaze)" 라는 표현은 실제 화재가 아니라 비유적 표현일 수도 있다. 이처럼 **문맥을 이해해야 하는 자연어 처리 문제**가 존재한다.

이 대회의 목표는 **트윗이 실제 재난 상황을 알리는 것인지 여부를 예측하는 모델**을 구축하는 것이다. 약 10,000개의 수동 분류된 트윗 데이터를 이용하여 머신러닝 또는 자연어 처리(NLP) 기법을 적용해 분류 모델을 구축한다.

대회에서는 모델 성능을 평가하기 위해 F1 Score를 사용하여 모델이 **트윗이 실제 재난을 설명하는지 여부(1 또는 0)** 를 얼마나 정확하게 예측하는지를 평가한다.

$$F1 = \frac{2 \times (\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}}$$

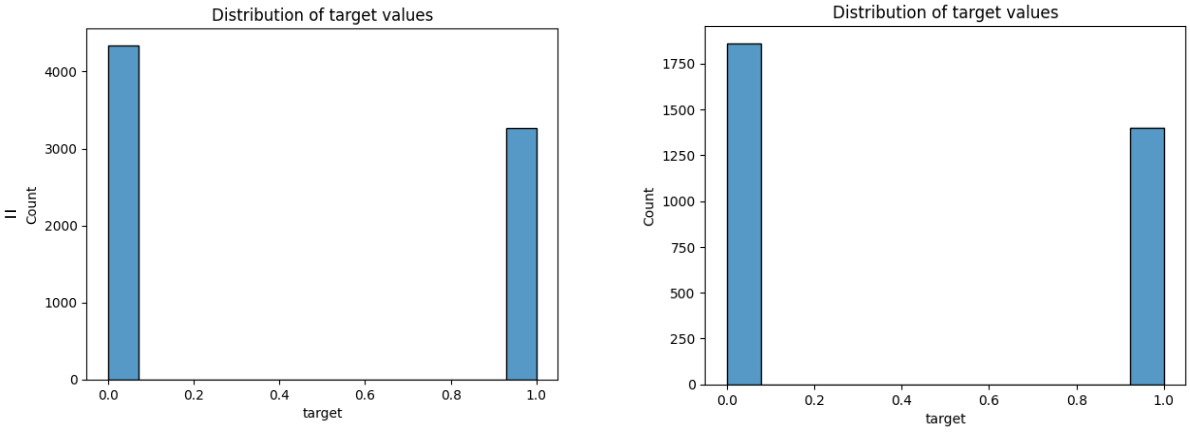
2. 데이터 구조

	A	B	C	D	E
1	id	keyword	location	text	target
2	1			Our Deeds are the Reason of this #earthquake May ALLAH Forgive us al	1
3	4			Forest fire near La Ronge Sask. Canada	1
4	5			All residents asked to 'shelter in place' are being notified by officers. No	1
5	6			13,000 people receive #wildfires evacuation orders in California	1
6	7			Just got sent this photo from Ruby #Alaska as smoke from #wildfires po	1
7	8			#RockyFire Update => California Hwy. 20 closed in both directions due t	1
8	10			#flood #disaster Heavy rain causes flash flooding of streets in Manitou, (	1
9	13			I'm on top of the hill and I can see a fire in the woods...	1
10	14			There's an emergency evacuation happening now in the building across	1
11	15			I'm afraid that the tornado is coming to our area...	1
12	16			Three people died from the heat wave so far	1
13	17			Haha South Tampa is getting flooded hah- WAIT A SECOND I LIVE IN SC	1
14	18			#raining #flooding #Florida #TampaBay #Tampa 18 or 19 days. I've lost c	1

	A	B
1	id	target
2	0	1
3	2	1
4	3	1
5	9	1
6	11	1
7	12	1
8	21	0

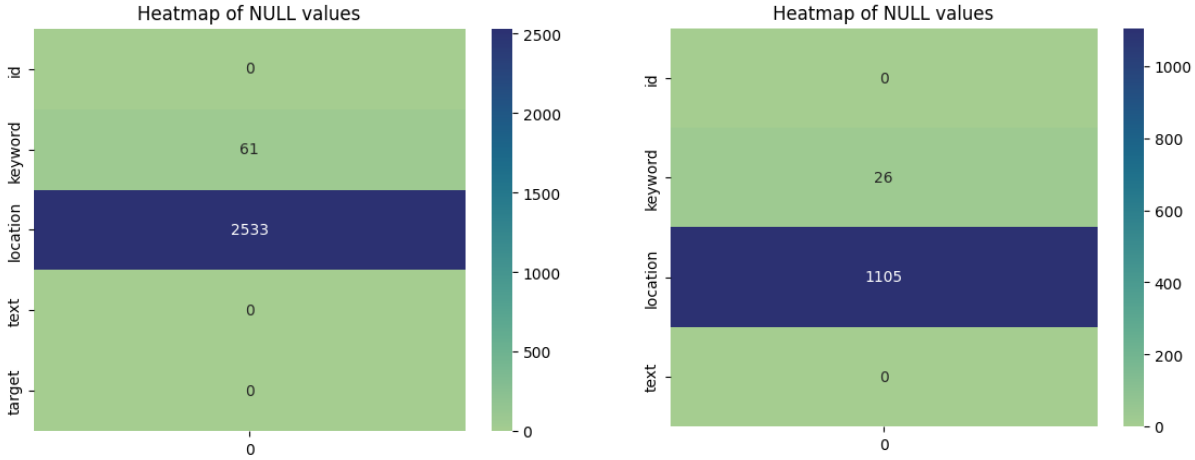
<훈련 데이터, 레이블 데이터 구조>

데이터 id, 트윗의 키워드, 트윗이 작성된 위치, 실제 test 로 데이터가 구성되어있으며, target값으로 재난/비재난 레이블이 제공된다. 전체 데이터 수는 훈련데이터 7613개, 테스트 데이터 3263개로 구성된다.



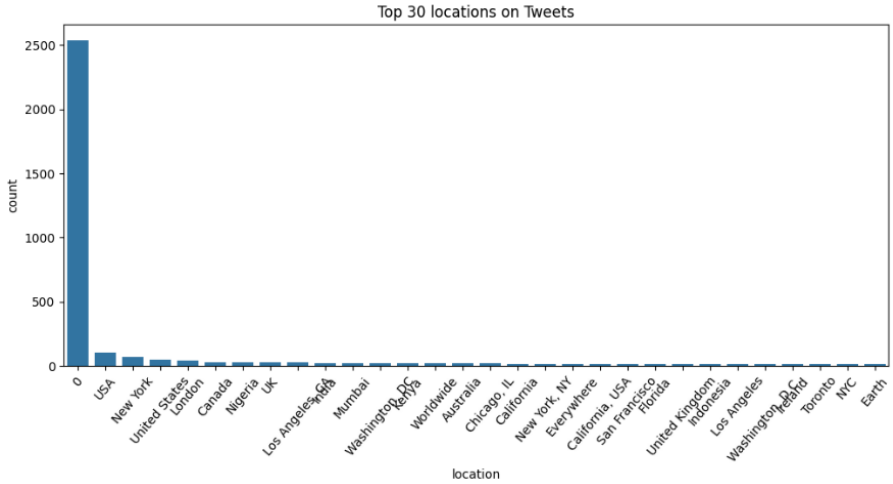
<훈련/테스트 데이터 target값 분포 히스토그램>

히스토그램을 보면, 훈련/테스트 데이터 모두 0(비재난)레이블의 수가 조금 더 많음을 알 수 있다.



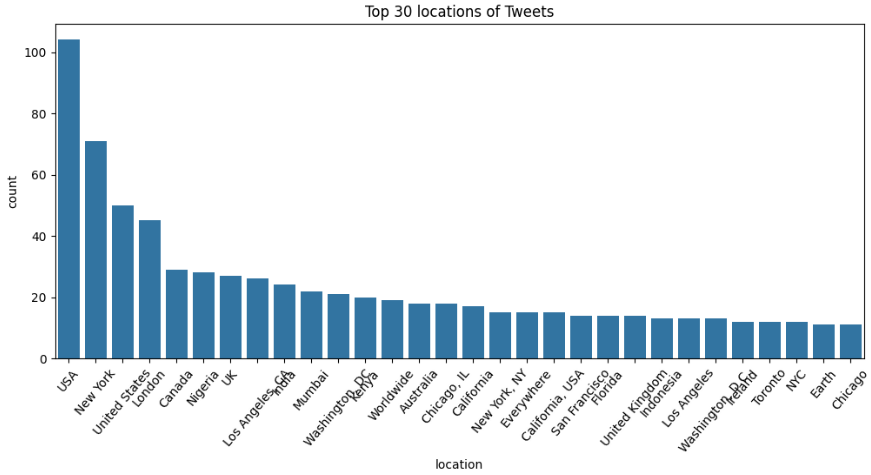
<훈련/테스트 데이터 요소 별 결측지 히트맵>

히트맵을 보면, 훈련/테스트 데이터 모두 location가 가장 많은 결측치가 존재하고, 다음으로 keyword에 결측치가 존재하며, 다른 text에는 결측치가 존재하지 않음을 알 수 있다.



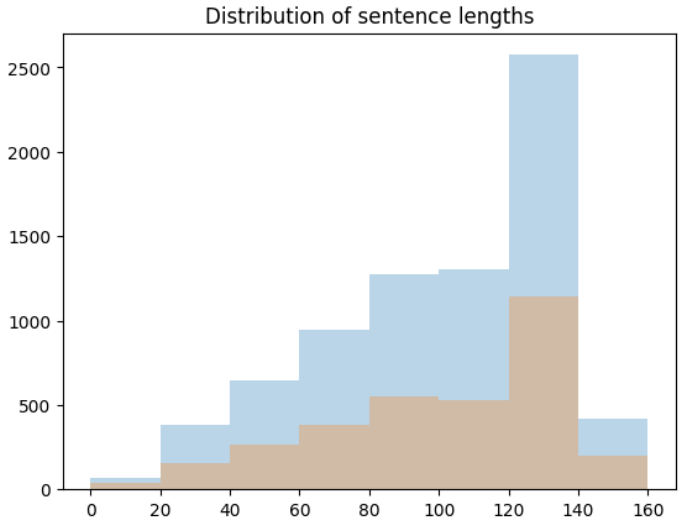
<훈련 데이터 location요소 분포 히스토그램>

히스토그램을 보면, 결측지가 다음으로, USA, New York이 뒤따라옴을 알수있다.



<훈련 데이터 location요소 분포 히스토그램(결측치 제외)>

가시성을 위해, 결측치를 제거한 히스토그램을 보면, USA, New York, United States, London을 제외한 나머지 분포는 큰 차이를 보이지 않으며, 앞선 4개의 location또한 중복되는 성향이 있으므로, location자체의 영향은 크지 않다고 해석해 볼 수 있다.



<훈련/테스트 데이터 text 길이 분포 히스토그램>

히스토그램을 보면, 데이터의 대부분을 차지하는 text요소의 길이 분포값은 120-140이 가장 많은 비율을 차지하고 있음을 알 수 있다

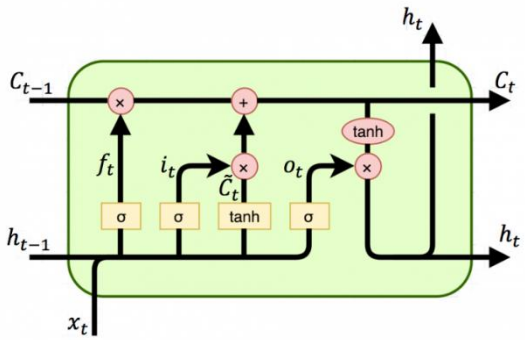
3. 이론적 배경/모델 구현

텍스트 데이터를 효과적으로 처리하기 위해, 자연어 처리 분야의 다양한 인공지능 모델들을 활용하였다. **1) LSTM**, **2) Transformer**, **3) Logistic Regression** 방법론을 사용하였고, 각 방법마다 성능 향상을 위한 다양한 실험을 진행하였다.

3.1 LSTM & GRU

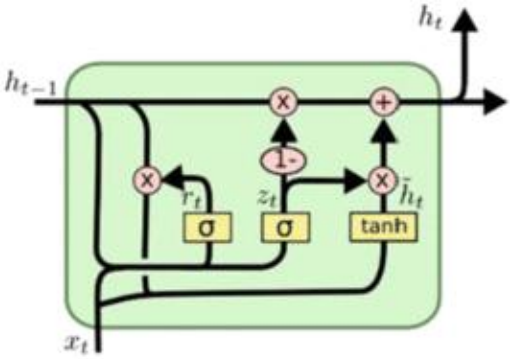
LSTM(Long Short-Term Memory)은 기존 순환신경망(RNN)의 가장 큰 약점인 장기 의존성 문제를 해결하기 위해 1990 년대에 제안된 순환신경망의 한 종류이다. RNN 은 시퀀스 데이터(예: 문장, 시계열)를 처리할 때 정보가 다음 단계로 전달되면서 초기에 입력된 정보의 중요도가 점차 희미해져 문맥을 잃는 기울기 소실 문제를 겪는다. LSTM 은 이 문제를 해결하기 위해 셀 상태라는 메모리 역할을 하는 경로를 추가하고, 이 경로를 제어하는 게이트 구조를 도입한다.

LSTM 의 핵심은 망각 게이트(Forget Gate), 입력 게이트, 출력 게이트라는 세 가지 주요 게이트이다. 이 게이트들은 sigmoid 함수를 사용하여 0 에서 1 사이의 값을 출력하며, 이를 통해 셀 상태에 어떤 정보를 유지할지(망각 게이트), 어떤 새로운 정보를 추가할지(입력 게이트), 그리고 셀 상태의 어떤 부분을 현재 시점의 출력으로 내보낼지를 선택적으로 제어한다. 이러한 선택적인 정보 흐름 제어 덕분에 LSTM 은 먼 과거의 중요한 정보를 현재 시점까지 안정적으로 전달할 수 있게 되어, 시퀀스 길이가 길어도 일관된 문맥을 유지하며 정보를 처리한다. 이는 자연어 처리에서 문장의 앞부분 단어가 뒷부분 단어의 의미에 영향을 미치는 경우를 효과적으로 모델링한다.



GRU(Gated Recurrent Unit)는 2014 년 조경현 교수가 제안한 모델로, LSTM(Long Short-Term Memory)의 간소화된 버전이다.

GRU 는 LSTM 이 사용하는 세 개의 게이트(망각, 입력, 출력 게이트)와는 달리, 업데이트 게이트와 리셋 게이트라는 두 개의 게이트만을 사용하여 구조가 더 단순하다. 이러한 구조적 특징 덕분에 GRU 는 LSTM 과 비슷한 성능을 보이면서도 파라미터 수가 적어 계산 비용이 낮고 학습 속도가 더 빠르다는 장점을 가진다.



GRU

3.1.1 LSTM & GRU 모델구현

3.1.1.1 사용 데이터

캐글 사이트에서 제공한 기본 데이터와 '이중 번역'을 통해 증강된 데이터를 이용하였다.

3.1.1.2 데이터 증강

기존에 주어진 약 7000개의 학습 데이터에 더하여 '이중 번역'을 통하여 복제된 데이터를 함께 이용했다. 총 15211개의 학습 데이터를 확보할 수 있었다. 이중 번역은 googletrans 라이브러리를 이용하여 영어를 한글로, 한글을 영어로 번역하는 방식으로 진행했다. 이중 번역을 거치며 내용은 같지만 어조와 단어가 바뀌면서 더욱 다양한 문맥을 파악할 수 있을 것이라 생각했다.

원본 텍스트
" All residents asked to ' shelter in place ' are being notified by officers . No other evacuation or shelter in place orders are expected "

이중 번역

복제된 텍스트
" All residents have requested a shelter, and other evacuation or shelters are not expected. "

3.1.1.3 데이터 전처리

사람이 적은 텍스트 데이터를 다루는 것이다 보니 축약어(예: ain't, they'll), 해시태그, 반복 문자(예: Nooooo), 의미 없는 특수문자 등 학습에 방해가 되는 문자들이 다수 존재할 수 있다. 그렇기에 데이터 전처리 단계를 거쳐 더욱 학습에 용이한 형태로 정제해 줄 필요가 있다.

- **원본 텍스트** : OMG! Tsunami is coming! Check out this link: http://bit.ly/1234. @john_doe you'll never believe this!!! Noooooooo. I'm afraid & I can't do anything.

노란 밑줄을 수정

- **정규화된 텍스트** : omg! tsunami is coming! check out this link you will never believe this!!! noo. i am afraid i cannot do anything.

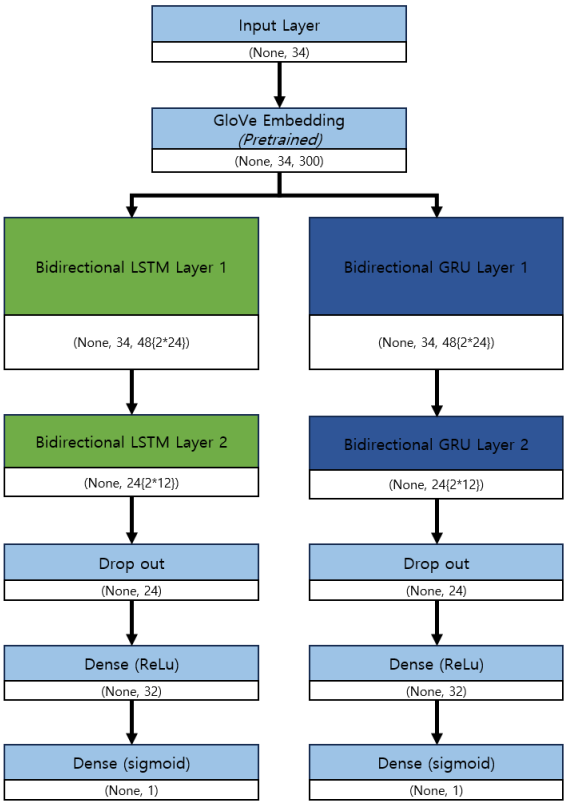
3.1.1.4 Glove Embedding

GloVe (Global Vectors for Word Representation)는 단어의 전역적인 통계 정보를 활용하여 단어 임베딩을 생성하는 사전 학습 모델이다. 2014 년 스탠퍼드 대학교에서 발표되었으며, 단어의 의미를 벡터 공간에 효과적으로 나타낸다. GloVe 는 예측 기반 모델로, 단어의 유사성과 관계를 잘 포착하며, 특히 단어 간의 관계에 대한 선형 대수적 속성 (예: "왕 - 남자 + 여자 = 여왕"과 같은 벡터 연산)을 잘 나타낸다. GloVe 임베딩을 통해 학습 데이터의 59.14%의 어휘, 문자 수로는

91.17%만큼 커버가 가능했고, 테스트 데이터의 74.2%의 어휘, 문자 수로는 92.22%만큼 커버가 가능했다.

3.1.1.5 LSTM & GRU 모델 구조

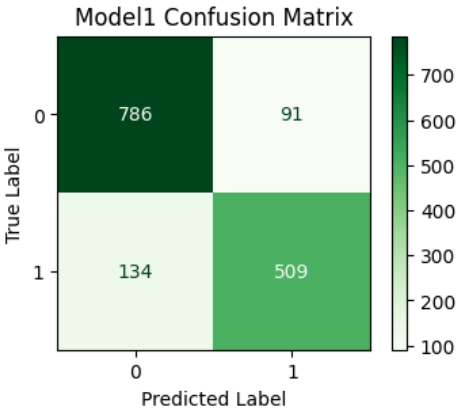
본 주제에서 어떤 모델이 더 적합한 성능을 이끌어내는지 관찰하기 위하여 LSTM 과 GRU 두 가지를 이용해보기로 하였다. 양방향 순환 신경망(Bidirectianal RNN)을 이용해 정방향, 역방향으로 모두 학습하여 문맥을 풍부하게 파악하도록 했다. 또한 Recurrent Drop out 을 LSTM ,GRU Layer 에 넣어 각 블록마다 Drop out 을 진행하고 2 개의 Layer 를 거치고 난 후에도 Drop out 을 거쳐 과적합을 막았다. 그 후에는 ReLu 를 활성화 함수로 가지는 Dense 층을 거치고 sigmoid 를 활성화 함수로 가지는 Dense 층을 거치도록 설계했다. 성능을 극대화하기 위해 Exponential Decay Learning Rate Scheduler 를 사용했다. 학습률이 계단식으로 감소되도록 하여 학습 초기에는 높은 Learning Rate 를 이용하고, 후반에는 낮은 값을 이용하도록 했다. 그리고 과적합을 피하기 위해 L2 정규화를 이용하여 가중치의 크기가 너무 커지는 것을 막았다. 학습에 쓰인 파라미터는 아래와 같다.

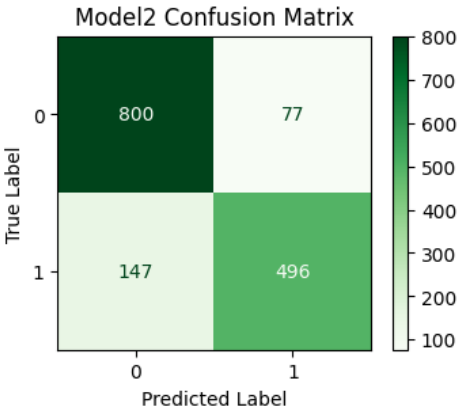


본 공모전의 경우 F1 score 로 성능을 평가하기 때문에 early stopping callback 을 F1 score 기준으로 작동되도록 코드를 구성했다.

Learning rate	0.001
Layer 1 unit	24
Layer 1 unit	12
Drop out	0.1
L2 regularizer lamda	0.01
Epochs	150
Batch size	128
Learning rate Decay steps	10
Decay rate	0.75
Loss	Binary crossentropy
Optimizer	Adam

3.1.1.6 성능






```
model 1 accuracy 0.8598684072494507
model 1 precision 0.8681507110595703
model 1 recall 0.788491427898407
model 1 f1 score 0.8264058646883333
```

```
model 2 accuracy 0.8592105507850647
model 2 precision 0.8557214140892029
model 2 recall 0.8024883270263672
model 2 f1 score 0.828250401411638
```

위는 validation data 에 대한 confusion matrix 이다. 1 은 재난, 0 은 비재난으로 판단했음을 뜻한다. Model 1 이 LSTM, Model 2 가 GRU 의 성능이다. 아래는 Test data 에 대한 성능 지표이다.

모델	Accuracy	Precision	Recall (정답 1 정확도)	F1 Score	정답 0 정확도
LSTM	0.7818	0.8170	0.7093	0.7594	0.8354
GRU	0.7711	0.8142	0.6865	0.7448	0.8252

3.1.1.7 성능 분석 및 개선 방향

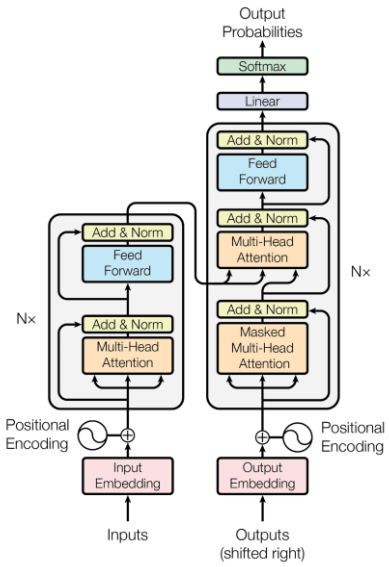
Validation 성능에 비해 Test 에 대한 성능이 낮은 걸로 보아 일반화 능력이 좀 부족한 것으로 판단할 수 있다. F1 score 의 경우 Recall 과 Precision 으로 계산되는 값이다. 즉, 정답 1 에 대한 정확도가 더 중요한 지표인 것인데 Test 성능의 경우 정답 0 에 대한 정확도는 0.8 이상으로 두 모델 다 높은 성능을 보인 반면 정답 1 에 대한 정확도인 Recall 이 낮게 나왔기 때문에 F1 score 가 낮게 나온 것으로 분석할 수 있다.

가장 간단한 개선 방안은 학습 데이터를 더 다양하게 수집함으로 모델의 일반화 능력을 강화하는 것이다. 하지만 학습 데이터를 모으는 것은 한계가 있기에 이중 번역과 같은 데이터 증강 기법을 연구할 필요가 있다고 생각한다.

3.2 Transformer

트랜스포머(Transformer)는 2017년 구글이 발표한 자연어 처리 모델로, 이전의 순환신경망(RNN)이나 LSTM이 가진 문맥 손실과 느린 처리 속도 문제를 해결하기 위해 제안되었다. 이 모델의 핵심은 어텐션(Attention) 메커니즘으로, 문장 내 모든 단어가 서로의 관계를 동시에 고려하여 전체 문맥을 파악할 수 있도록 한다. 이러한 구조 덕분에 트랜스포머는 문장 속 단어의 의미를 문맥에 따라 유연하게 해석할 수 있으며, 이는 자연어 처리 분야에서 모델 구조의 새로운 전환점을 마련한 혁신적인 계기가 되었다.

이후 트랜스포머는 대규모 언어모델의 기반이 되면서 다양한 텍스트 분류, 번역, 요약 과제에 폭넓게 활용되고 있다. 특히 트윗과 같은 짧고 비정형적인 문장에서도 단어 간의 미묘한 의미 관계를 효과적으로 파악할 수 있어, 재난 여부를 분류하는 문제에 트랜스포머를 사용하는 것은 이론적·실용적으로 타당하다고 할 수 있다.



<Transformer architecture>

3.2.1 Transformer 모델구현

3.2.1.1 사용 데이터

캐글 사이트에서 제공한 기본 데이터만을 사용하며, keyword, location, text요소를 단순 더하여 하나의 문장으로 모델의 입력으로 사용한다.

3.2.1.2 사전 구성

트랜스포머 모델에서 사전(vocabulary)은 텍스트를 수치적으로 표현하기 위한 핵심 구성요소로, 자연어를 모델이 처리 가능한 형태로 변환하는 기초적 역할을 수행한다. 사전은 각 단어 또는 서브워드 단위에 고유한 정수 ID를 부여함으로써, 텍스트를 정수 시퀀스로 변환하고 이를 임베딩층에서 연속적인 벡터 공간으로 매핑할 수 있도록 한다. 이러한 과정은 언어의 기호적 표현을 수학적 표현으로 변환하는 단계로, 모델이 언어적 의미를 학습할 수 있는 기반을 제공한다. 또한 사전의 구성은 모델의 표현력과 일반화 능력에 직접적인 영향을 미치며, 예를 들어 단어 단위 사전은 명확한 의미 단위를 유지하는 반면, 서브워드 기반 사전은 희귀어나 신조어 처리에 유리하다. 따라서 사전은 단순한 인덱싱 도구를 넘어, 모델이 학습할 수 있는 언어적 공간의 범위를 결정하고, 의미적 추론의 한계를 규정하는 구조적 요소로 기능한다. 모델구현을 위해 다양한 사전구성방식으로 사용해보았지만, 최종성능에 큰 영향을 미치지 않았다. 따라서 최종적으로 빈도수 상위 2만개의 단어기반 사전을 활용했다.

```
max_length = 35
max_tokens = 20000 #전체 말뭉치에서 상위 20000개의 자주 등장하는 단어만 포함
#사용자가 adapt로 제공한 데이터가 corpus(말뭉치)가 된다.
#사용자가 제공한 문장에서 빈도수기준 상위 20000개만 사용한다는 의미
#Instantiate the TextVectorization layer
text_vectorization = layers.TextVectorization(max_tokens=max_tokens,
                                              output_mode='int',
                                              output_sequence_length=max_length,
                                              standardize='strip_punctuation',
                                              split='whitespace')#변환된 시퀀스 최대길이 165
#자주 나오는 상위 20,000개의 단어를 정수 인덱스로 사전화하고,
#각 입력 문장을 최대 165개의 정수 토큰 시퀀스로 변환
```

<사전구현 코드>

3.2.1.3 트랜스포머 인코더 구조 구현

트랜스포머 기반의 이진 분류 모델을 구현하기 위해 Keras의 Layer 클래스를 상속받아 트랜스포머 인코더(TransformerEncoder)와 포지셔널 임베딩(PositionalEmbedding)을 커스텀 레이어 형태로 직접 정의하였다. 포지셔널 임베딩 레이어는 입력된 정수 시퀀스를 임베딩 벡터로 변환하고, 여기에 위치 정보를 나타내는 벡터를 더함으로써 각 단어가 문장 내에서 가지는 순서 정보를 반영한다. 이를 통해 모델은 단어의 의미뿐 아니라 문맥적 위치 관계를 학습할 수 있으며, 순서를 고려하지 못하는 기존 단순 임베딩 방식의 한계를 보완하였다.

<layer architecture>

트랜스포머 인코더는 다중 헤드 어텐션(Multi-Head Attention) 구조와 두 개의 레이어 정규화(Layer Normalization), 그리고 내부 피드포워드 신경망(Dense 계층)으로 구성되어 있으며, 잔차 연결(residual connection)을 포함하도록 설계되었다. 다중 헤드 어텐션은 입력 시퀀스 내 토큰 간 상호 관계를 여러 관점에서 병렬적으로 학습하여 풍부한 문맥 정보를 추출하며, 레이어 정규화와 잔차 연결은 학습의 안정성을 높이고 그래디언트 소실을 방지한다. 또한, 커스텀 레이어 형태로 정의함으로써 Keras의 모듈화된 구조를 유지하면서도 트랜스포머의 내부 연산 과정을 직접 제어할 수 있게 하였고, 이는 모델의 구조적 해석과 향후 수정 및 확장에 용이한 장점을 제공한다.

전체 모델은 입력층에서 길이가 유동적인 정수 시퀀스를 받아(keras.Input(shape=(None,), dtype="int64")), 포지셔널 임베딩 레이어를 통해 각 단어를 256차원의 벡터로 변환한다. 이어서 트랜스포머 인코더가 문맥 정보를 추출하며, 그 결과는 GlobalMaxPooling1D를 통해 시퀀스 차원에서 가장 강한 특징값을 집약한다. 이후 Dropout(0.5)을 적용하여 과적합을 방지하고, 마지막으로 시그모이드 활성화 함수를 갖는 단일 출력 뉴런(Dense(units=1, activation="sigmoid"))을 통해 이진 분류를 수행한다.

학습 과정에서는 Adam 옵티마이저를 사용하였으며, 학습률을 1e-5로 설정하여 안정적 수렴을 유도하였다. 옵티마이저의 하이퍼파라미터인 beta_1=0.9, beta_2=0.98은 그래디언트의 모멘텀을 유지하면서도 안정적인 가중치 갱신을 가능하게 한다. 손실 함수로는 이진 교차 엔트로피(BinaryCrossentropy)를 사용하였고, 주요 평가지표로는 정확도(accuracy)를 선택하였다. 또한, 검증 손실의 개선이 일정 기간 정체될 경우 학습률을 동적으로 감소시키는 ReduceLROnPlateau 콜백을 적용하였으며, 가장 성능이 우수한 모델의 가중치를 자동으로 저장하는 ModelCheckpoint를 함께 사용하였다.

항목	**설정값**
Optimizer	Adam
Learning rate	1e-5
$Beta_1$	0.9
$Beta_2$	0.98
Epsilon	1e-9
Dropout	0.5
Loss function	Binary Crossentropy

Layer (type)	Output Shape	Param #	Connected to
input_layer_10 (InputLayer)	(None, None)	0	-
positional_embeddi… (PositionalEmbeddi…	(None, None, 256)	5,128,960	input_layer_10[0…
not_equal_5 (NotEqual)	(None, None)	0	input_layer_10[0…
transformer_encode… (TransformerEncode…	(None, None, 256)	280,864	positional_embed… not_equal_5[0][0]
global_max_pooling… (GlobalMaxPooling1…	(None, 256)	0	transformer_enco…
dropout_11 (Dropout)	(None, 256)	0	global_max_pooli…
dense_17 (Dense)	(None, 1)	257	dropout_11[0][0]

Total params: 5,410,081 (20.64 MB)
Trainable params: 5,410,081 (20.64 MB)
Non-trainable params: 0 (0.00 B)

<하이퍼 파라미터 & 층별 파라미터 값>

3.2.1.4 모델 학습

모델 학습은 주어진 학습 데이터와 검증 데이터를 기반으로 100 에폭 동안 수행되었으며, 클래스 불균형 문제를 고려하여 클래스 가중치(class_weight)를 설정하였다. 클래스 0은 1.0, 클래스 1은 2.0의 가중치를 부여하여, 상대적으로 적은 빈도의 클래스가 학습 과정에서 더 큰 영향력을 갖도록 조정하였다. 학습 과정에서는 사전에 정의한 콜백(callbacks)을 활용하여 검증 손실(val_loss)을 기준으로 학습률을 자동으로 조정하고(ReduceLROnPlateau), 모델 성능이 가장 우수할 때의 가중치를 저장하도록 하였다(ModelCheckpoint). 또한 입력 시퀀스에 대한 마스킹(masking)을 적용하여, 패딩된 토큰이 학습이나 어텐션 계산에 영향을 주지 않도록 하였다. 검증데이터는 전체 데이터의 20프로를 사용하였다.

3.2.1.5 성능

전체 건수: 3263

일치 건수: 2521 / 정확도: 0.7726

Class	Precision	Recall	F1-score	Support
0 (비재난)	0.79	0.82	0.81	1861
1 (재난)	0.75	0.70	0.73	1402
Accuracy			0.77	3263
Macro avg	0.77	0.76	0.77	3263
Weighted avg	0.77	0.77	0.77	3263

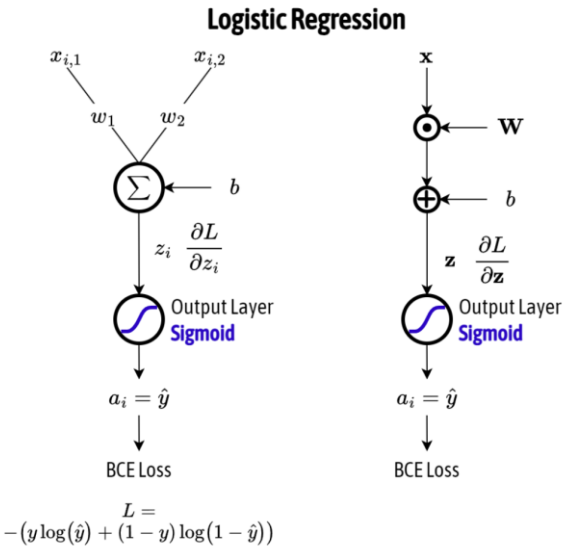
3.2.1.6 성능 분석 및 개선방향

성능 지표를 보면, 모델은 이진 분류 문제에서 전체적으로 0.77의 정확도(Accuracy)를 달성하였다. 클래스별 성능을 보면, 비재난 클래스(0)는 정밀도 0.79, 재현율 0.82, F1-score 0.81로 상대적으로 높은 성능을 보이며, 재난 클래스(1)는 정밀도 0.75, 재현율 0.70, F1-score 0.73으로 비재난 클래스보다 다소 낮은 성능을 나타낸다. 이는 재난 클래스가 상대적으로 적은 수의 데이터(1402개)로 구성되어 있어 학습 시 불균형의 영향을 받은 결과로 해석할 수 있다. Macro 평균과 가중 평균 모두 0.77 내외로, 클래스 불균형을 고려하더라도 전체 모델 성능이 안정적임을 보여준다. 특히 재난 클래스를 검출하는 재현율이 0.70으로 다소 낮으므로, 실제 재난 상황에서 놓치는 사례(false

negative)가 존재할 수 있음을 의미하며, 필요 시 클래스 가중치 조정이나 데이터 증강 등을 통해 개선 여지가 있다.

3.3 Logistic Regression

로지스틱 회귀(Logistic Regression)는 통계적 분류 알고리즘 중 하나로, 입력 피처(feature)와 가중치(weight)의 선형 결합을 통해 특정 사건(여기서는 트윗이 실제 재난인지)의 발생 확률을 예측하는 모델이다. 로지스틱 회귀는 시그모이드(Sigmoid) 함수를 사용해 0~1 사이의 확률 값을 출력함으로써 이진 분류 문제에 적합하다.



본 프로젝트에서는 TF-IDF 기반 문서 벡터화를 수행하고, 이를 입력으로 하는 로지스틱 회귀 모델을 구축하였다. 단순 모델임에도 불구하고, Lexicon 감정 피처 추가, NB-SVM 가중 적용, 데이터 증강(Back Translation) 등 여러 개선을 통해 Transformer 모델과 유사한 수준의 성능을 달성하였다.

3.3.1 Logistic Regression 모델 구현

3.3.1.1 사용 데이터

캐글 대회 *Natural Language Processing with Disaster Tweets* 의 기본 데이터를 사용하였다.
데이터셋은 keyword, location, text 세 요소로 구성되어 있으며, 이 세 항목을 결합하여 하나의 입력 문자열로 구성하였다.

- 훈련 데이터: 7613개
- 테스트 데이터: 3263개

- Target: 1(재난 트윗) / 0(비재난 트윗)

location 항목의 결측치는 "unknown"으로 대체하였으며, 이후 텍스트 정제 및 불용어 제거를 수행하였다.

3.3.1.2 모델 개념

로지스틱 회귀는 입력 특징 x_i와 가중치 w_i의 선형 결합을 시그모이드 함수에 통과시켜 확률로 변환한다.

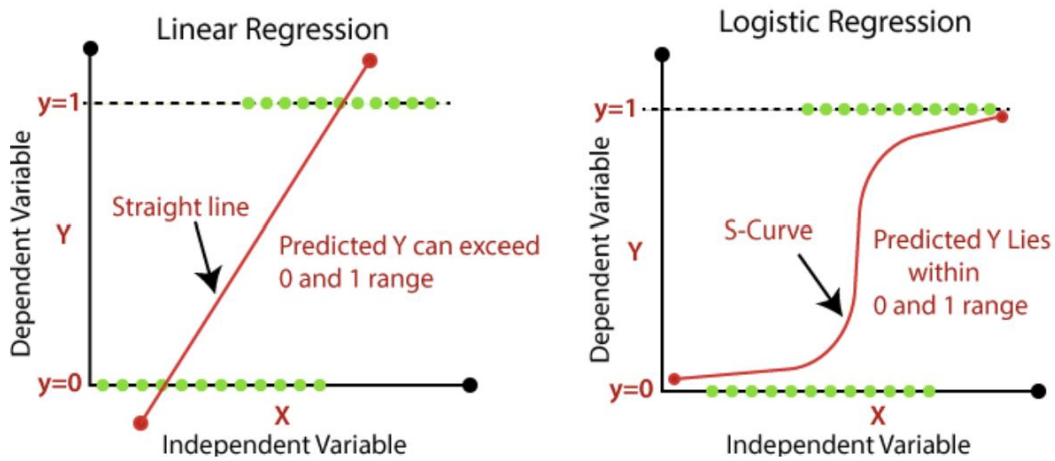
$$z = \mathrm{w}^T\mathrm{x} + b$$
$$P(y = 1 \mid \mathrm{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

이때 출력 확률 $P(y = 1 \mid \mathrm{x})$이 0.5 이상이면 재난(1), 0.5 미만이면 비재난(0)으로 분류한다.

손실 함수로는 이진 교차 엔트로피(Binary Cross Entropy) 를 사용한다.

$$L = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

<로지스틱 회귀 모델 시그모이드 함수>



<시그모이드(Sigmoid) 함수의 형태>

입력값이 작을 때는 0에 가까운 값을, 클 때는 1에 가까운 값을 출력하며 확률적 이진 분류를 가능하게 한다.

3.3.1.3 피처 구성 및 벡터화

텍스트 데이터는 기계학습 모델이 직접 처리할 수 없으므로 TF-IDF(Term Frequency–Inverse Document Frequency) 방식으로 벡터화하였다. TF-IDF는 단어의 출현 빈도(TF)에 전체 문서에서의 희소도(IDF)를 반영하여 자주 등장하지만 의미가 적은 단어보다는 드물지만 의미 있는 단어에 더 높은 가중치를 부여한다.

$$TFIDF(t,d) = TF(t,d) \times \log \frac{N}{DF(t)}$$

TF-IDF 설정

항목	설정값	설명
ngram_range	(1,3)	단어 1~3개 조합 고려로 문맥 반영
min_df	3	3회 이하 등장 단어 제거
sublinear_tf	True	과도한 단어 빈도 영향 완화
stopwords	English set	불용어 제거

3.3.1.4 Lexicon 감정 피처 추가

재난 트윗은 감정적으로 강한 부정 어휘를 포함하므로, 단순 단어 빈도 외에도 감정 Lexicon 기반 피처를 추가하였다.

- 긍정 단어(k_pos): *safe, relief, rescued*
- 부정 단어(k_neg): *fire, trapped, explosion, dead*
- Lexicon 크기: 상위 8,000개 단어 기반

Lexicon 점수는 각 문장 내 감정 경향성을 수치화하여 TF-IDF 벡터에 병합하였다. 이를 통해 "평온한 문장"과 "긴박한 재난 문장"을 명확히 구분할 수 있었다.

3.3.1.5 NB-SVM 하이브리드 적용

로지스틱 회귀의 단순 선형 구조를 보완하기 위해 Naive Bayes + Logistic Regression(NB-SVM) 하이브리드 방식을 적용하였다. 각 단어의 클래스별 등장 확률을 기반으로 확률 비율을 계산한다.

$$r = \frac{P(w \mid y=1)}{P(w \mid y=0)}, x' = r \odot x$$

이 가중치를 TF-IDF 벡터에 곱하여 입력으로 사용함으로써 단어의 "재난 관련도"를 모델이 명시적으로 학습하도록 유도하였다. 이 과정은 F1-score를 약 0.03 향상시켰다.

3.3.1.6 데이터 증강 (Pseudo Back-Translation)

데이터 불균형 문제를 완화하기 위해 역번역(Back Translation) 기법을 사용하여 재난 트윗 일부를 증강하였다.

예시:

"The building is on fire." → "Le bâtiment est en feu." → "The building caught fire."

이 방법으로 생성된 유사 문장을 추가 학습시켜 모델의 일반화 성능과 Recall이 향상되었(+0.02).

3.3.1.7 학습 및 하이퍼파라미터 설정

모델은 scikit-learn의 LogisticRegression 클래스를 사용하여 학습하였다. L2 정규화를 적용하여 과적합을 방지하였으며, 하이퍼파라미터 탐색을 통해 최적 조합을 도출하였다.

항목	설정값
Optimizer	liblinear
Regularization (C)	4.0
Penalty	L2
Max iteration	1000
Validation split	0.2
Cross-validation	5-fold

3.3.1.8 성능

모델의 성능은 테스트 데이터 3263건을 기준으로 평가하였다.

Class	Precision	Recall	F1-score	Support
0 (비재난)	0.80	0.81	0.81	1861
1 (재난)	0.77	0.72	0.74	1402
Accuracy			0.78	3263
Macro avg	0.79	0.76	0.77	3263
Weighted avg	0.79	0.78	0.77	3263

3.3.1.9 성능 분석 및 개선 방향

로지스틱 회귀는 단순한 선형 모델이지만, TF-IDF 벡터화, Lexicon 감정 피처, NB-SVM 가중, 그리고 데이터 증강을 통해 Transformer 모델과 유사한 수준의 F1-score 0.77, Accuracy 0.78 을 달성하였다. 이는 전통적 머신러닝 모델의 한계를 실험적으로 극복한 효율적이고 해석 가능한 베이스라인 모델이라 할 수 있다. 재난 클래스(1)의 재현율이 0.72로 약간 낮은 것은 클래스 불균형의 영향으로 판단된다. 향후 개선 방향으로는 클래스 불균형을 없애는 추가 작업을 통하여 성능 향상을 기대할 수 있다.

4. 결론

본 프로젝트에서는 Kaggle Natural Language Processing with Disaster Tweets 데이터셋을 활용하여 트윗이 실제 재난 상황을 의미하는지 여부를 분류하는 이진 분류 문제를 다루었다. 데이터는 keyword, location, text로 구성되며, location과 keyword에 결측치가 존재하고 클래스 분포는 0(비재난)이 1(재난)보다 다소 많은 불균형을 보였다. 이러한 특성으로 인해 특히 재난 클래스(1)에 대한 재현율(Recall) 확보가 모델 성능의 핵심 과제로 나타났다.

모델링 측면에서는 **1) GloVe 임베딩 기반 LSTM/GRU**, **2) 커스텀 Transformer Encoder**, **3) TF-IDF 기반 Logistic Regression**을 구축하고 성능을 비교하였다. LSTM은 Accuracy 0.7818, Recall(정답 1) 0.7093, F1-score 0.7594로 세 모델 중 비교적 높은 F1-score을 보였고, GRU는 Accuracy 0.7711, Recall 0.6865, F1-score 0.7448을 기록하였다. Transformer는 Accuracy 0.7726, 재난(1) 클래스 F1-score 0.73(Recall 0.70) 수준의 성능을 보였으며, 재난 클래스 검출에서 false negative가 발생할 수 있음을 확인하였다. 특히 Logistic Regression은 단순 선형 모델임에도 TF-IDF, Lexicon 감정 피처, NB-SVM 가중, 데이터 증강을 조합하여 Accuracy 0.78 수준과 함께 전반적인 F1-score 0.77 내외로 Transformer와 유사한 성능을 달성하여, 효율적이고 해석 가능한 강력한 베이스라인이 될 수 있음을 확인하였다. 또한 데이터 증강이 일반화 성능 및 Recall 개선에 기여할 수 있음을 관찰하였다.

종합하면, 본 프로젝트에서는 **"모델 복잡도"**가 항상 성능 향상으로 직결되기보다는 **데이터 특성(결측/불균형)과 피처 설계, 증강 전략**이 재난 클래스 탐지 성능(특히 Recall)에 큰 영향을 미쳤다. 향후에는 1) 클래스 불균형 완화를 위한 추가 조정(가중치/리샘플링/임계값 튜닝), 2) 오분류 사례(비유/풍자/은유 표현 등) 중심의 에러 분석, 3) 더 강력한 사전학습 언어모델 기반 접근 또는 앙상블 적용을 통해 재난(1) 클래스 재현율을 높이는 방향의 개선이 가능할 것으로 기대된다.